Name	**Raj Kalpesh Mathuria**
UID no.	**2023300139**

Experiment no. 6	
AIM :	Ensemble Learning using Boosting
Theory	Boosting is a supervised ensemble learning technique that combines multiple weak learners (usually decision trees) to form a strong and accurate predictive model. It works by training models sequentially, where each new model focuses on correcting the errors made by the previous ones. Unlike bagging methods, boosting assigns more importance to misclassified data points, allowing the model to learn difficult patterns more effectively. This results in improved accuracy and better generalization.

Code:	# ==========
	# BOOSTING ALGORITHMS EXPERIMENT
	# AdaBoost vs XGBoost
	# ==========
	
	import os
	import numpy as np
	import pandas as pd
	import matplotlib.pyplot as plt
	import seaborn as sns
	import joblib
	
	from sklearn.datasets import make_classification
	from sklearn.model_selection import train_test_split
	from sklearn.metrics import (
	  accuracy_score,
	  precision_score,
	  recall_score,
	  f1_score,
	  roc_auc_score,
	  confusion_matrix,
	  classification_report,
	  roc_curve
	)

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

from xgboost import XGBClassifier

#
============================================================
# Create folders
#
============================================================

os.makedirs("boosting_experiment/dataset", exist_ok=True)
os.makedirs("boosting_experiment/models", exist_ok=True)
os.makedirs("boosting_experiment/plots", exist_ok=True)
os.makedirs("boosting_experiment/metrics", exist_ok=True)

#
============================================================
# 1 Generate Synthetic Dataset
#
============================================================

X, y = make_classification(
    n_samples=5000,
    n_features=10,
    n_informative=6,
    n_redundant=2,
    n_classes=2,
    random_state=42
)

columns = [f"feature_{i}" for i in range(10)]

df = pd.DataFrame(X, columns=columns)
df["target"] = y

df.to_csv("boosting_experiment/dataset/synthetic_boosting_dataset.csv",
index=False)
```

```
print("Dataset saved.")

#
==============================================================
# 2 Train Test Split
#
==============================================================

X_train, X_test, y_train, y_test = train_test_split(
    df[columns], df["target"], test_size=0.2, random_state=42
)

#
==============================================================
# 3 Models
#
==============================================================

ada_model = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=200,
    learning_rate=0.5,
    random_state=42
)

xgb_model = XGBClassifier(
    n_estimators=200,
    max_depth=5,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss",
    use_label_encoder=False
)

#
==============================================================
# 4 Train Models
#
==============================================================
```

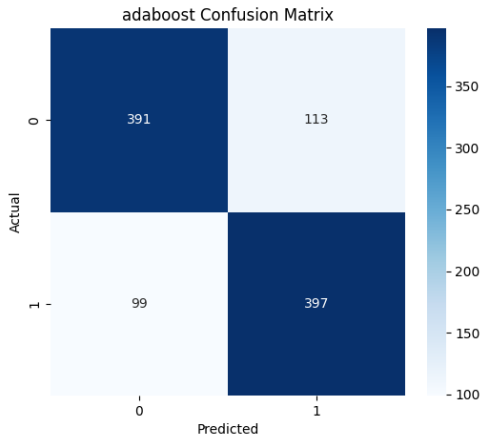
```
ada_model.fit(X_train, y_train)
xgb_model.fit(X_train, y_train)

# Save models
joblib.dump(ada_model, "boosting_experiment/models/adaboost_model.pkl")
joblib.dump(xgb_model, "boosting_experiment/models/xgboost_model.pkl")

#
============================================================
# 5 Predictions
#
============================================================

ada_pred = ada_model.predict(X_test)
xgb_pred = xgb_model.predict(X_test)

ada_prob = ada_model.predict_proba(X_test)[:,1]
xgb_prob = xgb_model.predict_proba(X_test)[:,1]

#
============================================================
# 6 Metrics Function
#
============================================================

def evaluate_model(name, y_true, y_pred, y_prob):

    acc = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred)
    recall = recall_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)
    auc = roc_auc_score(y_true, y_prob)

    return {
        "Model": name,
        "Accuracy": acc,
        "Precision": precision,
        "Recall": recall,
        "F1 Score": f1,
```

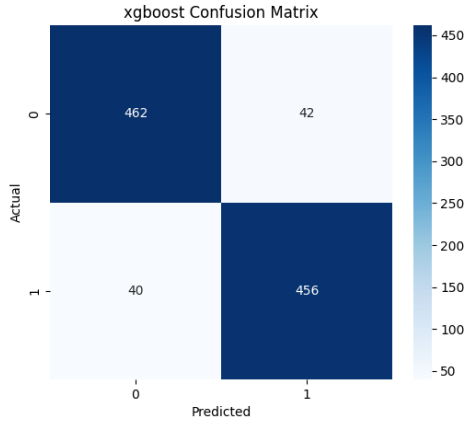
```
        "ROC-AUC": auc
    }

#
==============================================================
# 7 Compute Metrics
#
==============================================================

metrics = []

metrics.append(evaluate_model("AdaBoost", y_test, ada_pred, ada_prob))
metrics.append(evaluate_model("XGBoost", y_test, xgb_pred, xgb_prob))

metrics_df = pd.DataFrame(metrics)

metrics_df.to_csv("boosting_experiment/metrics/metrics_summary.csv",
index=False)

print(metrics_df)

#
==============================================================
# 8 Confusion Matrix Plot
#
==============================================================

def plot_confusion_matrix(y_true, y_pred, name):

    cm = confusion_matrix(y_true, y_pred)

    plt.figure(figsize=(6,5))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")

    plt.title(f"{name} Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")

    plt.savefig(f"boosting_experiment/plots/{name}_confusion_matrix.png")
    plt.close()
```

```
plot_confusion_matrix(y_test, ada_pred, "adaboost")
plot_confusion_matrix(y_test, xgb_pred, "xgboost")

#
============================================================
# 9 ROC Curve
#
============================================================

fpr_ada, tpr_ada, _ = roc_curve(y_test, ada_prob)
fpr_xgb, tpr_xgb, _ = roc_curve(y_test, xgb_prob)

plt.figure(figsize=(7,6))

plt.plot(fpr_ada, tpr_ada, label="AdaBoost")
plt.plot(fpr_xgb, tpr_xgb, label="XGBoost")

plt.plot([0,1],[0,1],'k--')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")

plt.legend()

plt.savefig("boosting_experiment/plots/roc_curves.png")
plt.close()

#
============================================================
# 10 Feature Importance
#
============================================================

importance = xgb_model.feature_importances_

plt.figure(figsize=(8,5))
sns.barplot(x=importance, y=columns)
```

```
plt.title("XGBoost Feature Importance")

plt.savefig("boosting_experiment/plots/feature_importance.png")
plt.close()

# ============================================================
# 11 Model Comparison Plot
# ============================================================

metrics_df.set_index("Model").plot(kind="bar", figsize=(10,6))

plt.title("Boosting Model Comparison")

plt.savefig("boosting_experiment/plots/model_comparison.png")
plt.close()

print("Experiment Complete.")
```

OUTPUT:





Model | Accuracy | Precision | Recall | F1 Score | ROC-AUC

AdaBoost | 0.788 | 0.77843137254902 | 0.800403225806452| 0.789264413518887| 0.876656105990783
XGBoost | 0.918 | 0.91566265060241 | 0.919354838709677| 0.917505030181086| 0.976314484126984

CONCLUSION:

The experiment shows that boosting algorithms significantly improve model performance by focusing on correcting previous errors. Among the models, XGBoost clearly outperforms AdaBoost, achieving higher accuracy, precision, recall, F1-score, and ROC-AUC. The confusion matrix also shows that XGBoost makes fewer misclassifications compared to AdaBoost. This indicates that advanced boosting techniques with optimization and regularization provide better generalization and robustness. AdaBoost performs reasonably well but struggles more with errors compared to XGBoost. Overall, boosting methods, especially XGBoost, are highly effective for classification tasks and provide superior predictive performance.

Code link: https://github.com/CodeCraftsmanRaj/ML_Sem6/tree/main/Exp6

Submit on Moodle before the Deadline and in PDF format only.